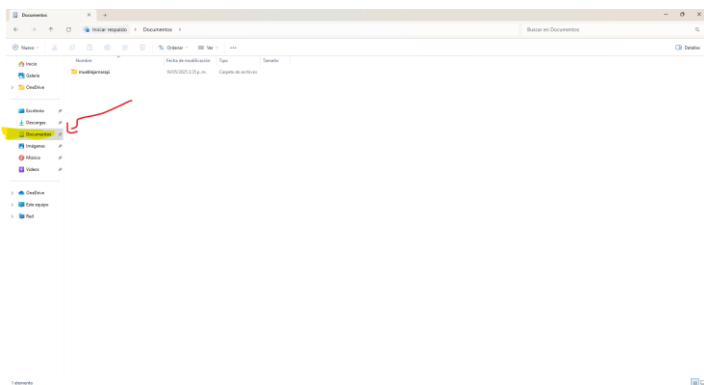
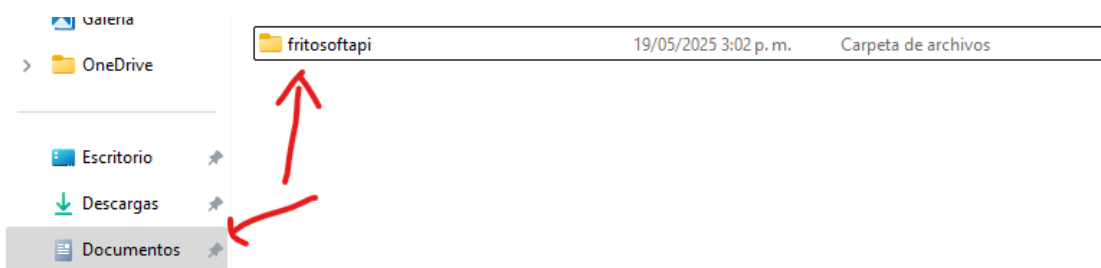


1 paso

Crear en directorio documento la carpeta raíz (crear lo mas posible pegado a la letra del disco)

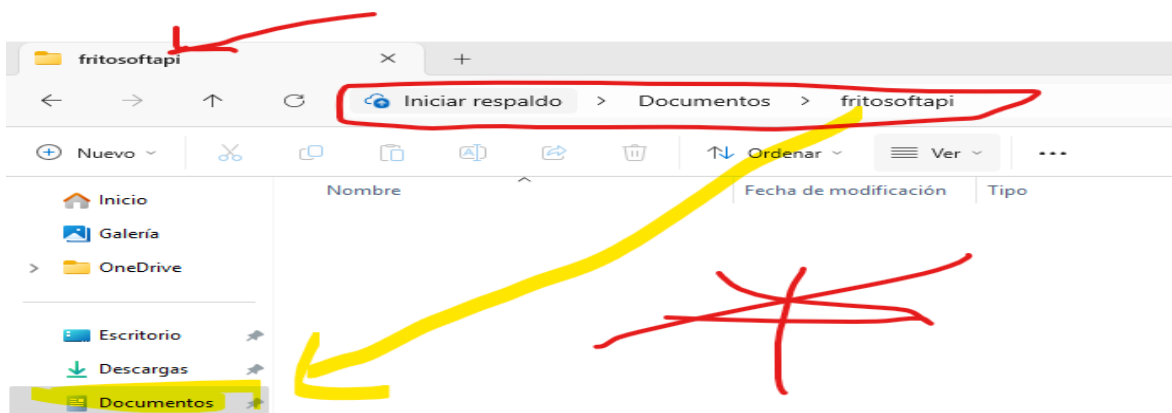


Nombre del archivo fritosoftapi debe ser minúscula sin caracteres especiales

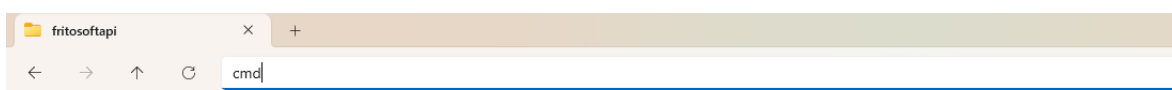


2 paso

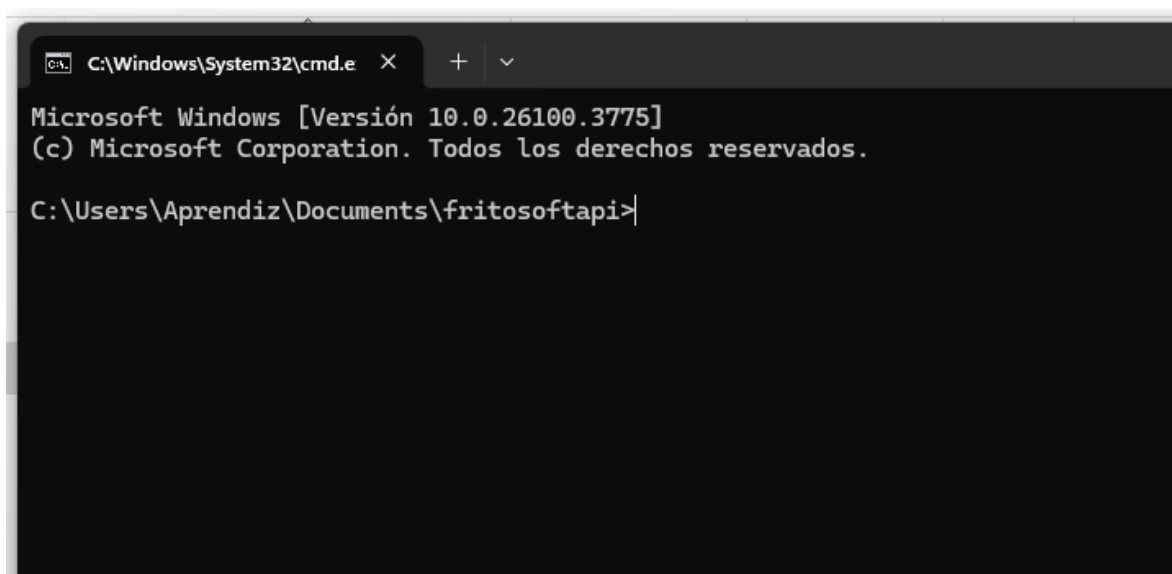
Entrar a la carpeta fritosoft (raíz)



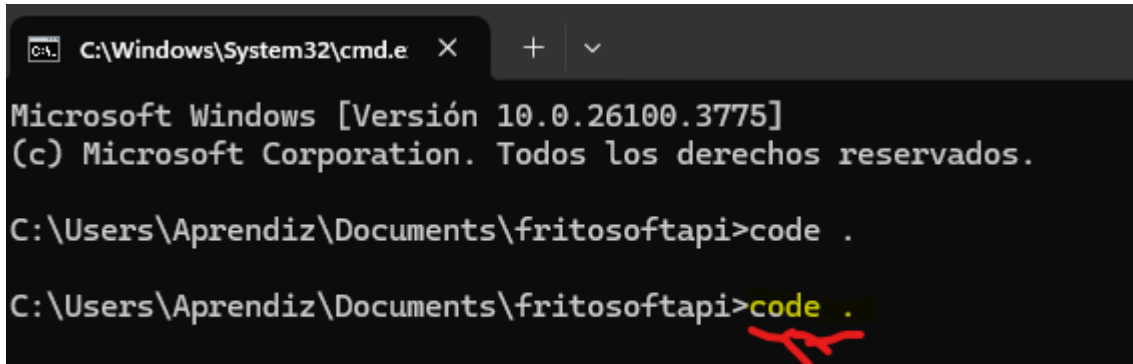
Escribir cmd en la dirección ->Enter



Les va salir una ventana así.



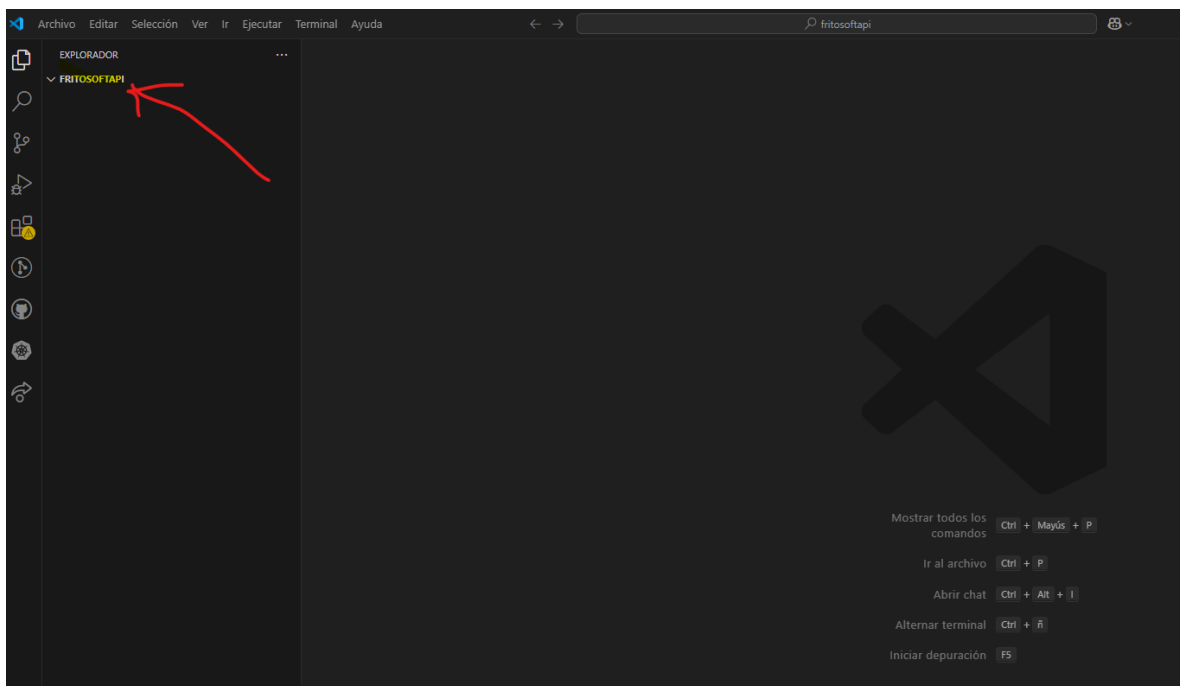
En esta ventana escriben code .



```
C:\Windows\System32\cmd.e X + v  
Microsoft Windows [Versión 10.0.26100.3775]  
(c) Microsoft Corporation. Todos los derechos reservados.  
C:\Users\Aprendiz\Documents\fritosoftapi>code .  
C:\Users\Aprendiz\Documents\fritosoftapi>code .
```

A red arrow points from the text "En esta ventana escriben code ." to the second instance of the `code .` command in the terminal.

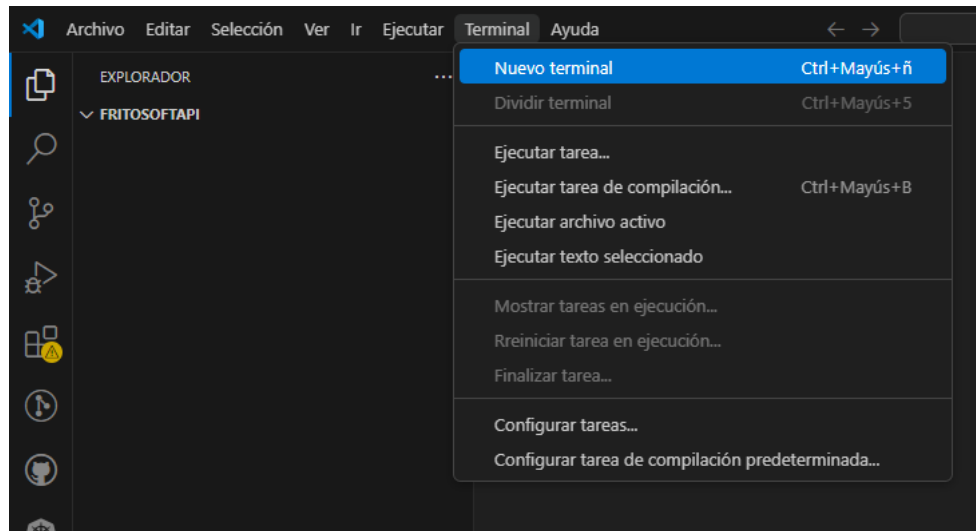
Para que les muestre el editor visual code vinculando la carpeta



3 paso

En este paso vamos a instalar node, pero debemos sacar la terminal les mostrare 2 pasos

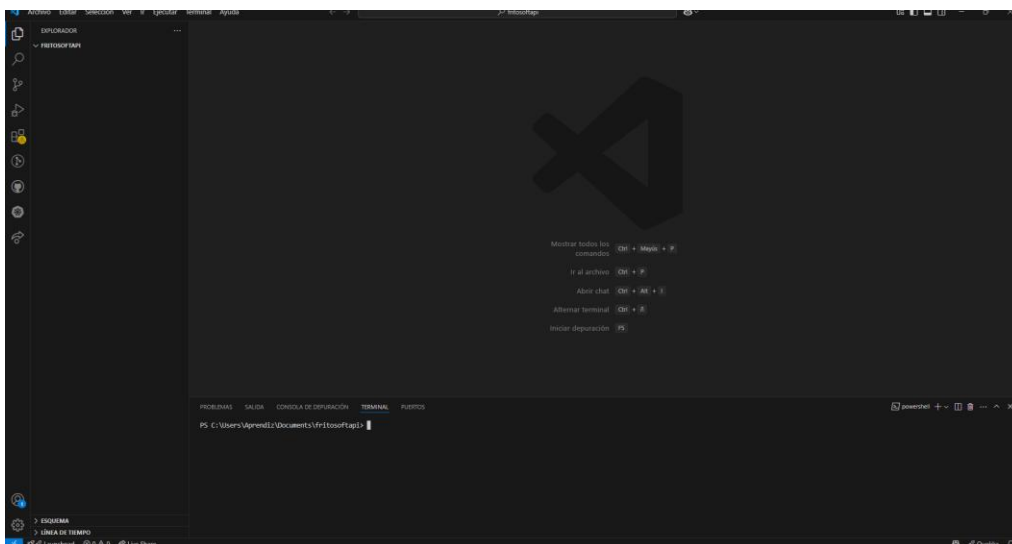
Paso a forma manual menú terminal nuevo terminal

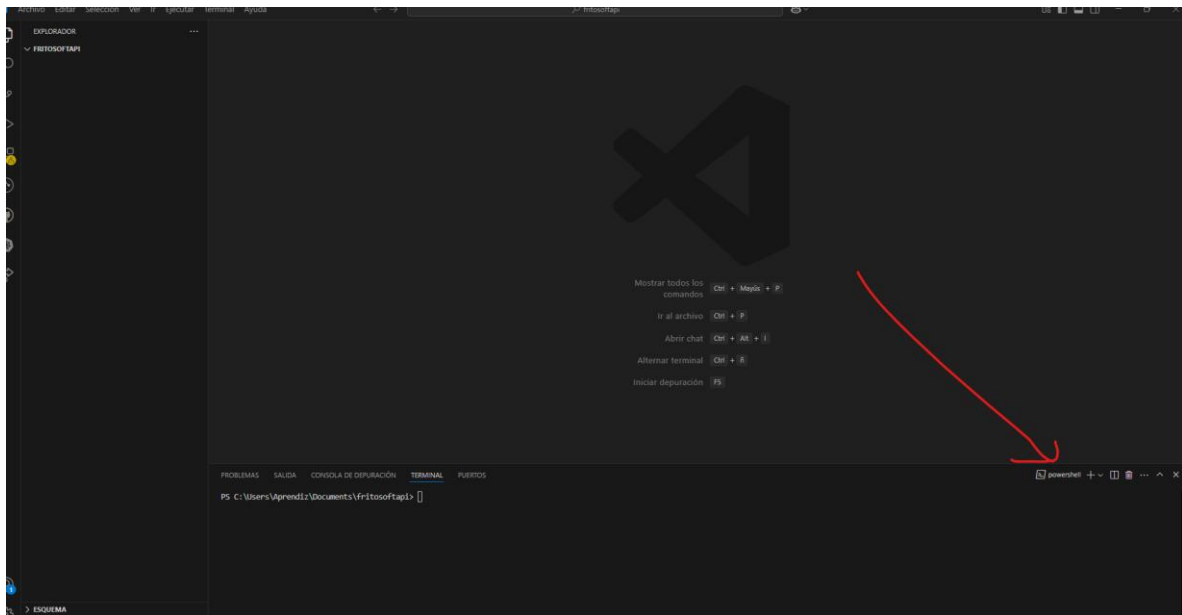


Paso b

Aquí solo precionamos al mismo tiempo ctrl + shif
+ ñ

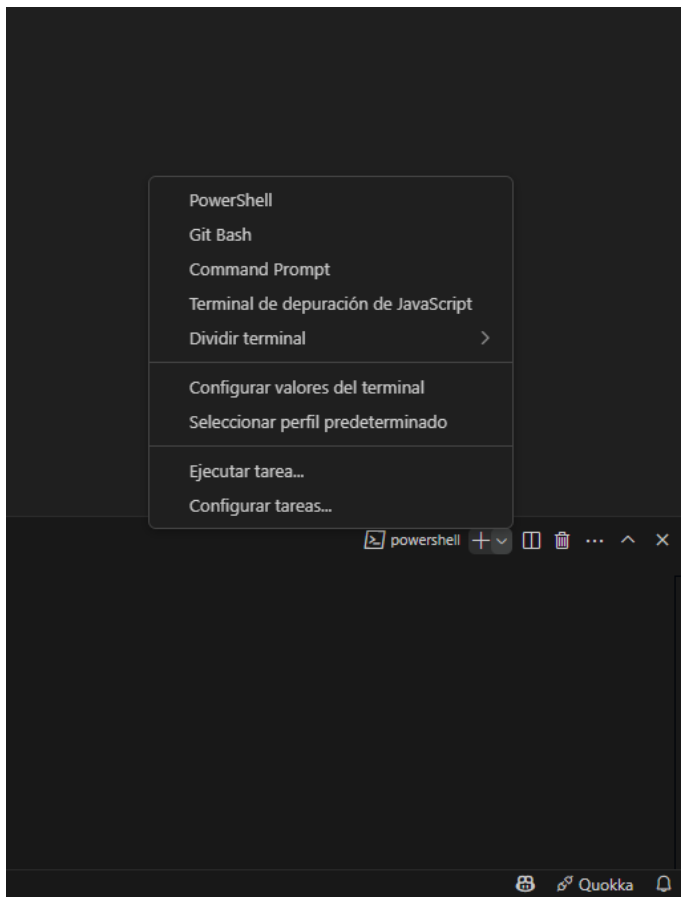
Las dos forma nos mostrara asi



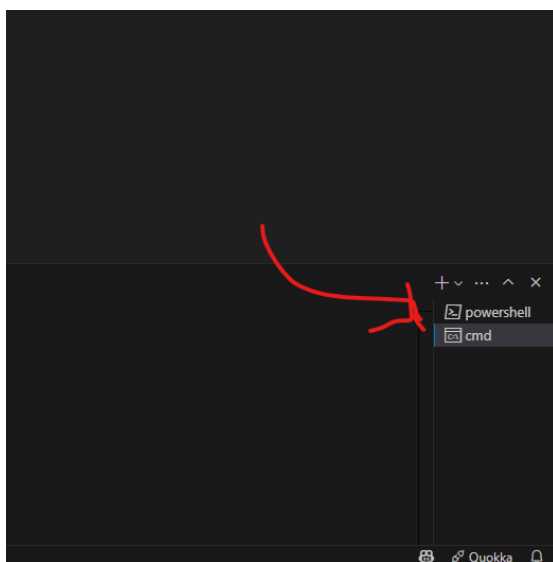


Esto que esta mostrando la flecha es el terminal powerShell si no tenemos permiso nos mostrara una serie de errores

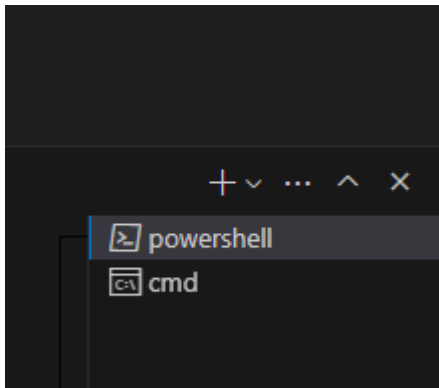
La idea es que lo configuremos a promand comand



En la uña del lado del + mas le damos clic para que nos aparezca esa ventana y podamos tomar la opción Comand prompt

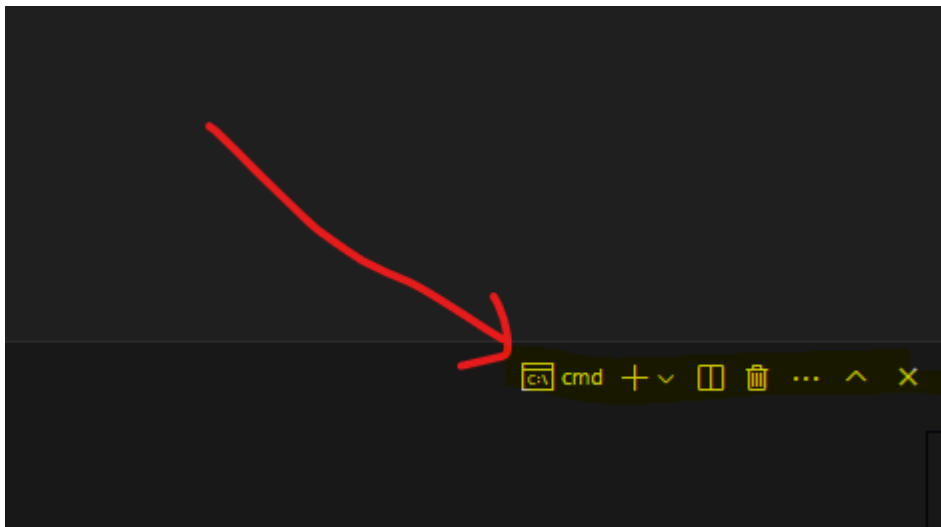


Eliminamos la otra PowerShell por futuros errores si esta ahí.



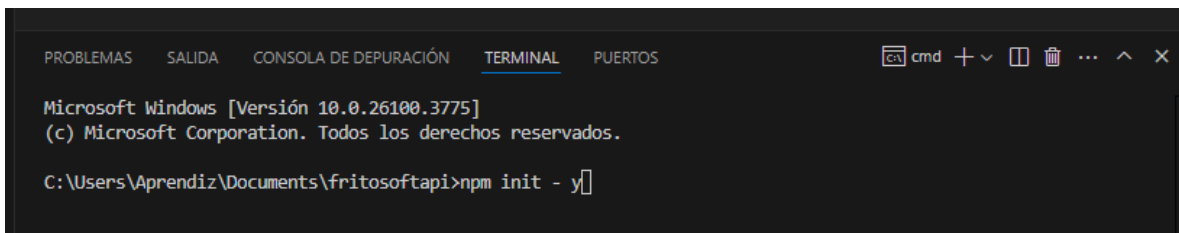
Al final de el si nos paramo saldrá un bote de basura le damos clic y eso lo elimina

Nos quedaría así

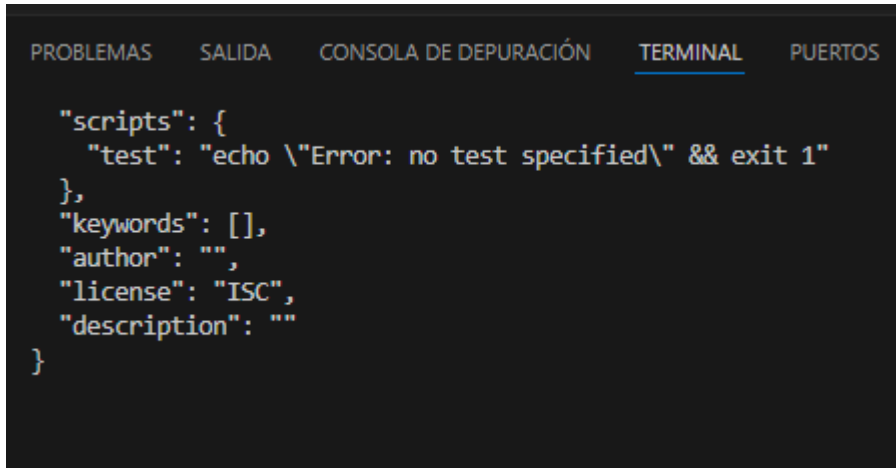


Ahora si vamos a instalar node

Utilizamos el comando npm init -y



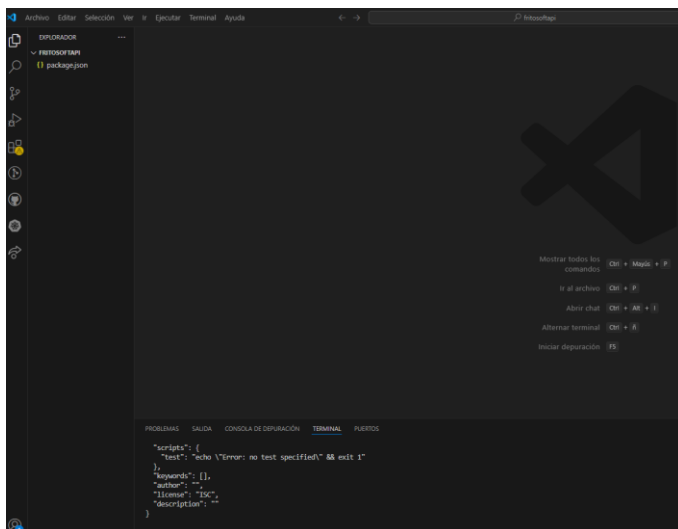
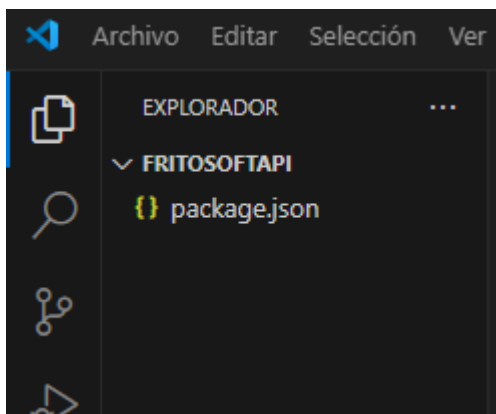
Al darle enter en la consola muestra así



```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1"
},
"keywords": [],
"author": "",
"license": "ISC",
"description": ""
}
```

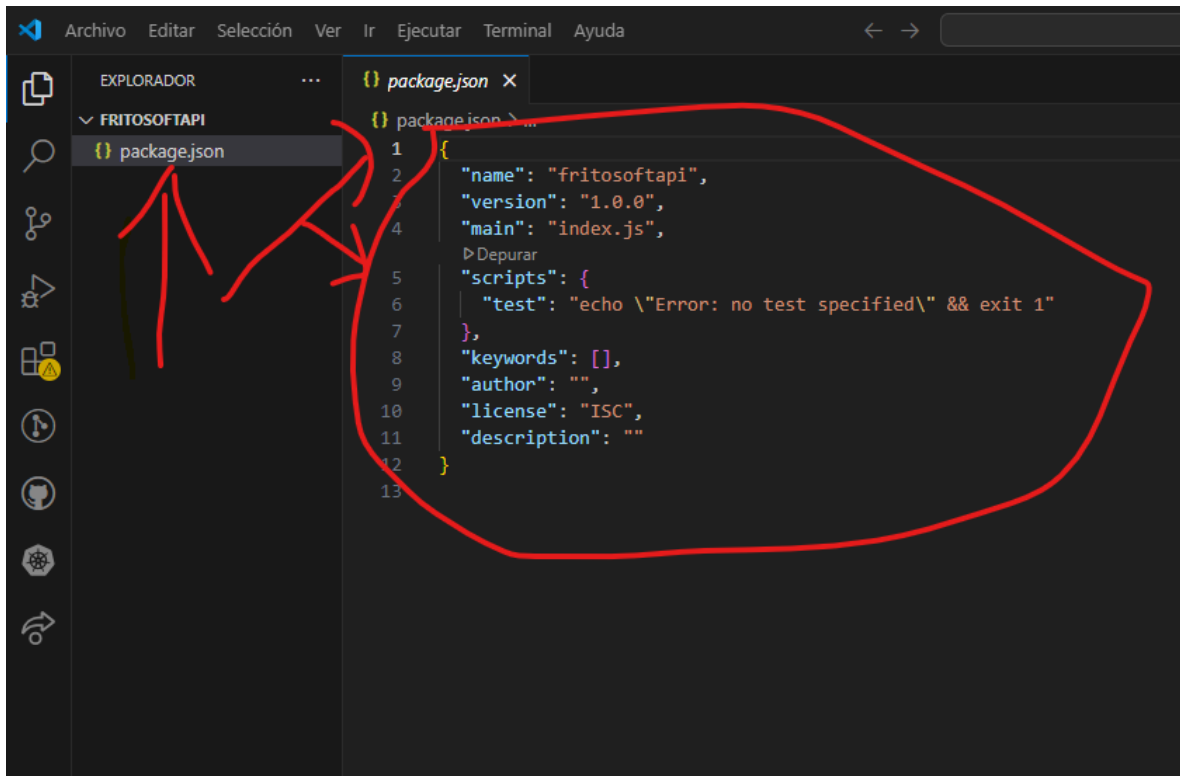
Y en el árbol así



Con esto quedo instalado node

3 paso

Configuración del package.json



The screenshot shows the Visual Studio Code interface with the Explorer sidebar on the left displaying a project named 'FRITOSOFTAPI' containing a 'package.json' file. The main editor area shows the 'package.json' file with the following content:

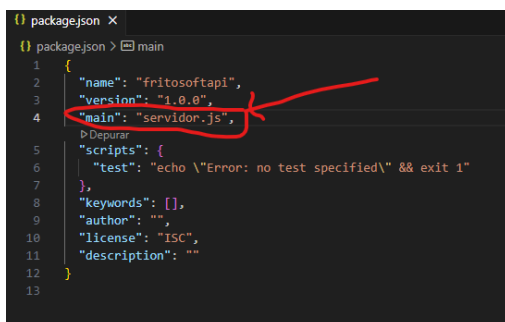
```
1 {  
2   "name": "fritosoftapi",  
3   "version": "1.0.0",  
4   "main": "index.js",  
5   "scripts": {  
6     "test": "echo \\\"Error: no test specified\\\" && exit 1"  
7   },  
8   "keywords": [],  
9   "author": "",  
10  "license": "ISC",  
11  "description": ""  
12 }  
13
```

Red annotations include a circle around the entire file content and arrows pointing from the Explorer sidebar to the file name 'package.json' and from the file name to the opening curly brace of the JSON object on line 1.

En la línea 4 línea 6

Línea 4

"main": "index.js", esta línea colcamos el nombre que llevara el archivo principal el nombre que yo le voy a dar es servidor.js



This close-up screenshot shows the 'package.json' file with the 'main' field on line 4 highlighted by a red circle and a red arrow pointing to it. The content of the file is as follows:

```
1 {  
2   "name": "fritosoftapi",  
3   "version": "1.0.0",  
4   "main": "servidor.js",  
5   "scripts": {  
6     "test": "echo \\\"Error: no test specified\\\" && exit 1"  
7   },  
8   "keywords": [],  
9   "author": "",  
10  "license": "ISC",  
11  "description": ""  
12 }  
13
```

Línea 6

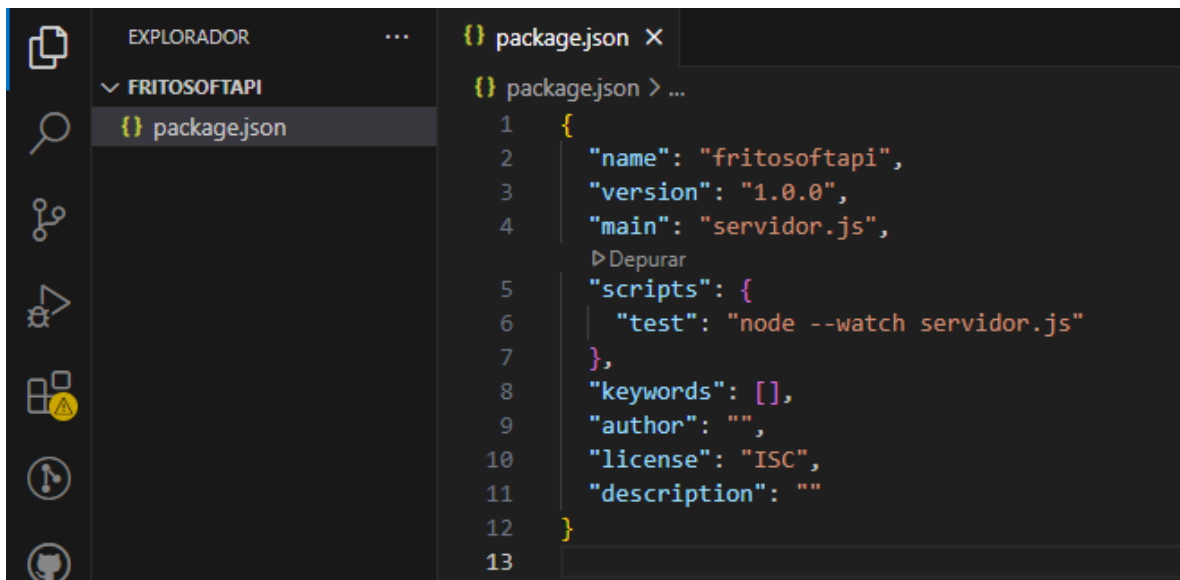
`"test": "echo \"Error: no test specified\" && exit 1"`

Esta línea las primeras comillas definen el tipo son tres dev desarrollo y test como esta es para testear y prd producción cuando este listo.

En este caso será dev

Y las segunda comilla es que servidor voy a utilizar para mi servidor node y el nombre del archivo principal

Quedando así `"dev": "node --watch servidor.js"`



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar shows a project named 'FRITOSOFTAPI' with a file named 'package.json' selected. The main editor area displays the content of 'package.json' with the following code:

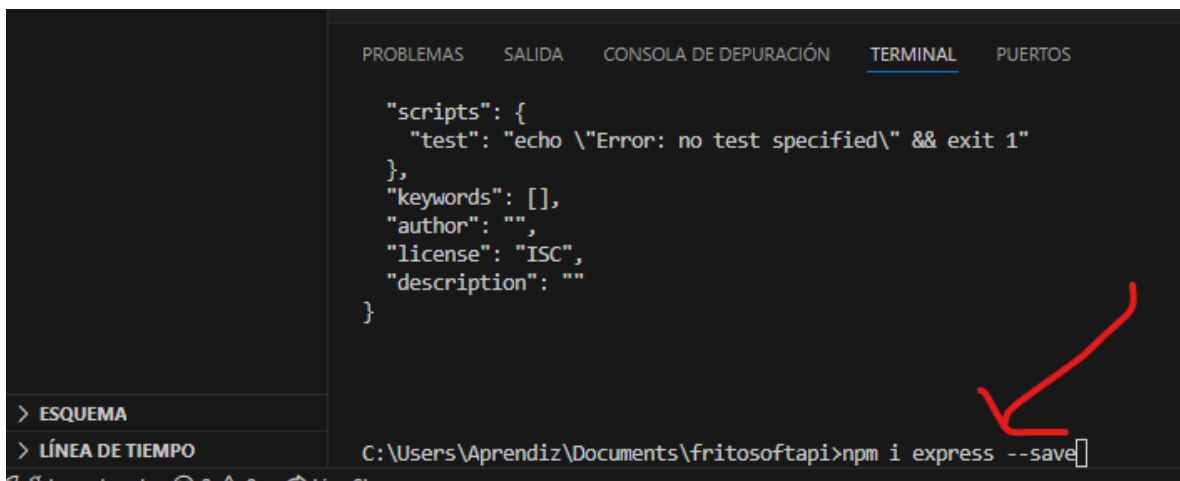
```
1  {
2    "name": "fritosoftapi",
3    "version": "1.0.0",
4    "main": "servidor.js",
5    "scripts": {
6      "test": "node --watch servidor.js"
7    },
8    "keywords": [],
9    "author": "",
10   "license": "ISC",
11   "description": ""
12 }
13
```

4 paso

Instalación de librerías

Express esta librería Express.js es un framework web minimalista y flexible para Node.js que simplifica el desarrollo de aplicaciones web y APIs. Es como una herramienta que te ayuda a estructurar y gestionar la lógica de tu servidor, sin que tengas que preocuparte por los detalles más bajos de Node.js.

Código es `npm i express --save`



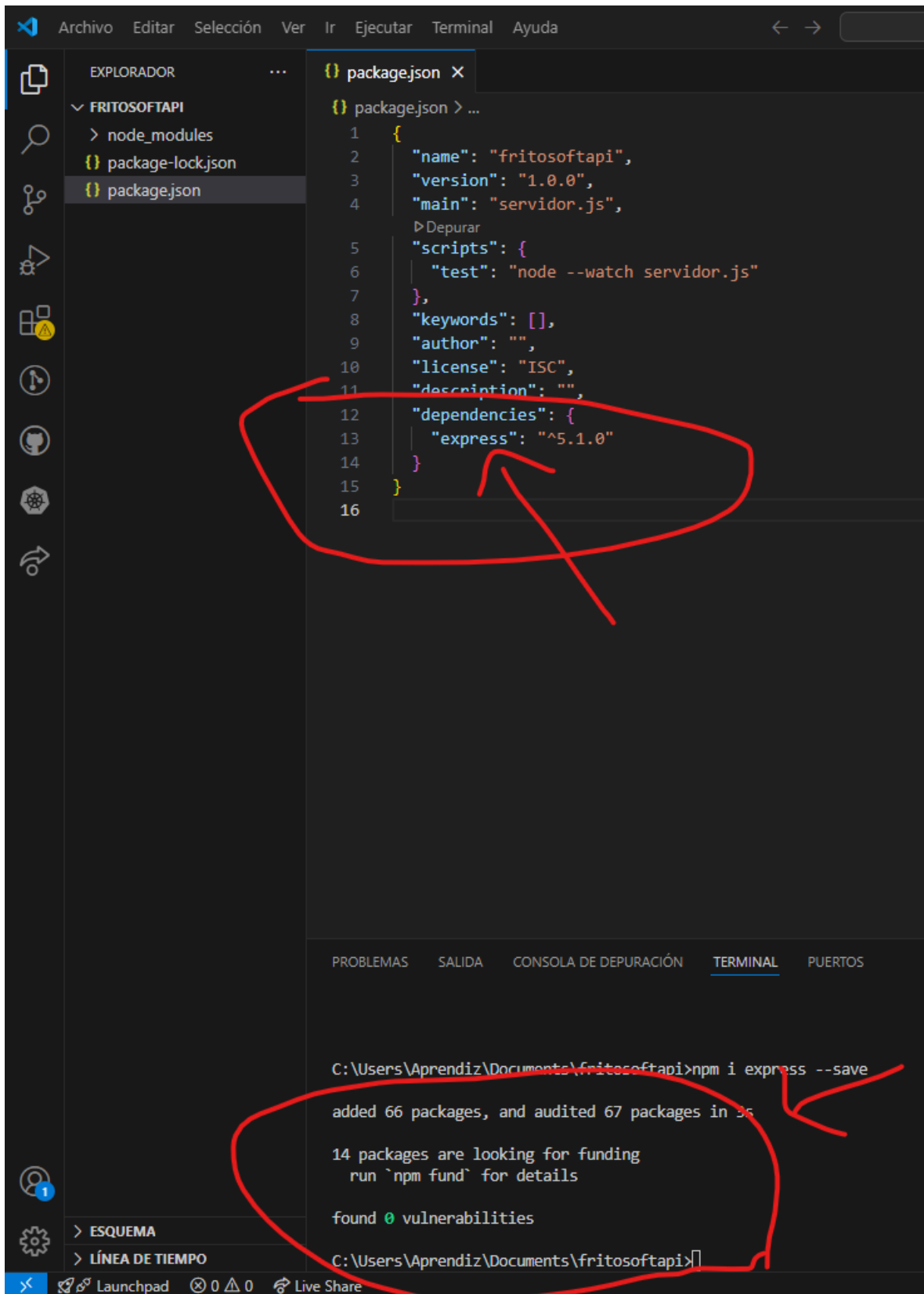
The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar is open, showing a file named 'package.json'. The main editor area displays the content of 'package.json', which includes a 'scripts' section with a 'test' script, and fields for 'keywords', 'author', 'license', and 'description'. Below the editor, the Output window is open, showing the command 'C:\Users\Aprendiz\Documents\fritosoftapi>npm i express --save' being executed in the terminal. A red arrow points from the terminal output to the 'package.json' file in the Explorer sidebar.

```
"scripts": {  
  "test": "echo \"Error: no test specified\" && exit 1"  
},  
"keywords": [],  
"author": "",  
"license": "ISC",  
"description": ""  
}
```

> ESQUEMA
> LÍNEA DE TIEMPO

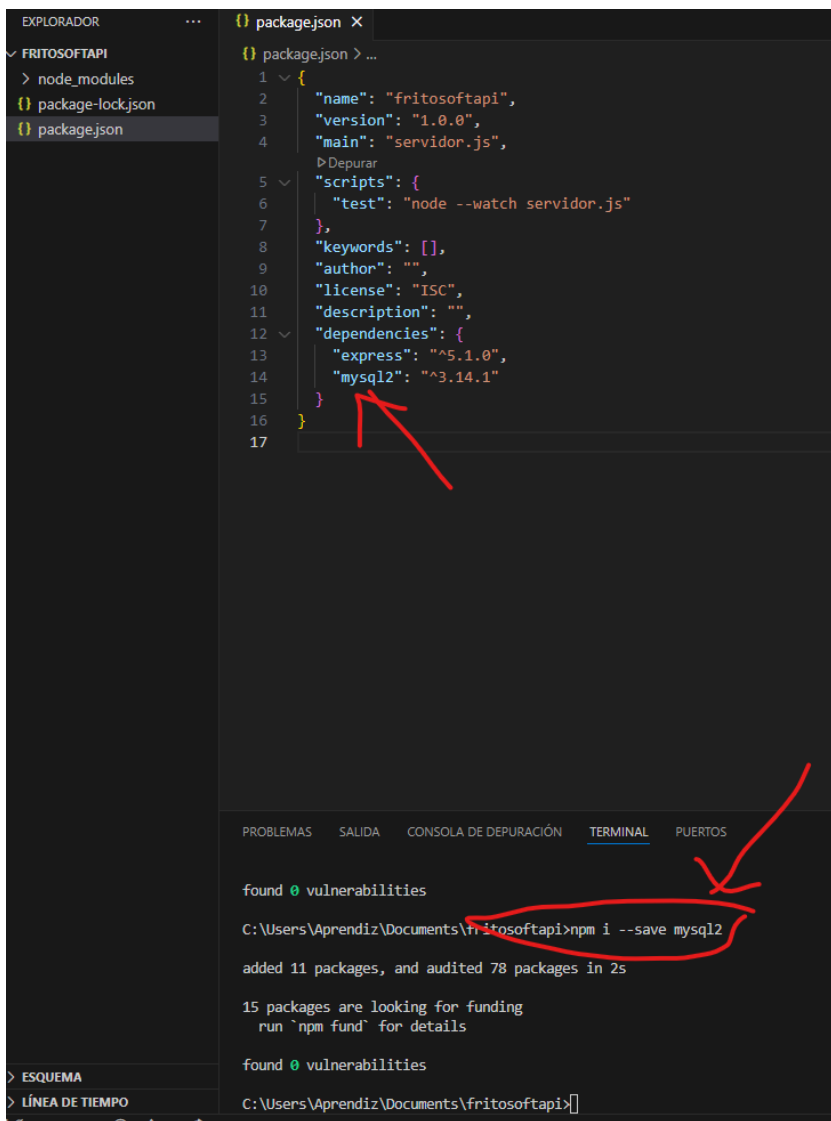
C:\Users\Aprendiz\Documents\fritosoftapi>npm i express --save

Con esto se instala verificamos así



mysql2 es un paquete para Node.js que facilita la interacción con bases de datos MySQL. Permite realizar conexiones, consultas, y otras operaciones SQL dentro de tus aplicaciones Node.js.

código npm i - - save mysql2



The image shows a screenshot of the Visual Studio Code editor. On the left, the Explorer sidebar shows the project structure with 'FRITOSOFTAPI' containing 'node_modules', 'package-lock.json', and 'package.json'. The 'package.json' file is open in the editor, showing the following content:

```
1 {  
2   "name": "fritosoftapi",  
3   "version": "1.0.0",  
4   "main": "servidor.js",  
5   "scripts": {  
6     "test": "node --watch servidor.js"  
7   },  
8   "keywords": [],  
9   "author": "",  
10  "license": "ISC",  
11  "description": "",  
12  "dependencies": {  
13    "express": "^5.1.0",  
14    "mysql2": "^3.14.1"  
15  }  
16 }  
17
```

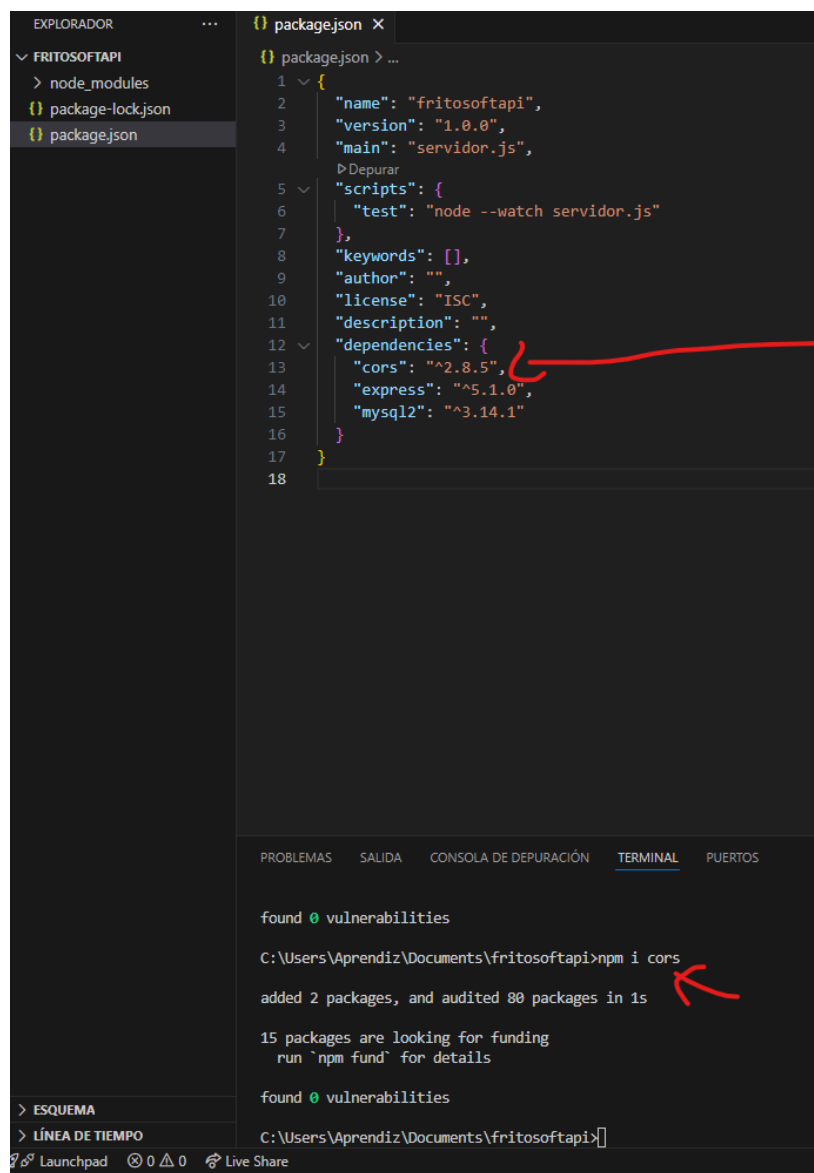
A red arrow points to the 'mysql2' dependency in the 'dependencies' object. Below the editor, the Terminal panel is open, showing the output of the command 'npm i --save mysql2'. The output includes:

```
found 0 vulnerabilities  
C:\Users\Aprendiz\Documents\fritosoftapi>npm i --save mysql2  
added 11 packages, and audited 78 packages in 2s  
  
15 packages are looking for funding  
run `npm fund` for details  
  
found 0 vulnerabilities  
C:\Users\Aprendiz\Documents\fritosoftapi>
```

A red circle highlights the command 'npm i --save mysql2' in the terminal, and a red arrow points to it from the right.

CORS, que significa "Cross-Origin Resource Sharing" o "Intercambio de Recursos de Origen Cruzado", es un mecanismo de seguridad web que permite a los navegadores controlar qué orígenes pueden acceder a los recursos de un servidor. En Node.js, CORS se utiliza para permitir que aplicaciones web que se ejecutan en un dominio accedan a recursos en otro dominio.

Código npm i cors



The screenshot shows a VS Code editor with a file explorer on the left and a code editor in the center. The file explorer shows a project named 'FRITOSOFTAPI' with files 'node_modules', 'package-lock.json', and 'package.json'. The 'package.json' file is open in the code editor, showing the following content:

```
1 {  
2   "name": "fritosoftapi",  
3   "version": "1.0.0",  
4   "main": "servidor.js",  
5   "scripts": {  
6     "test": "node --watch servidor.js"  
7   },  
8   "keywords": [],  
9   "author": "",  
10  "license": "ISC",  
11  "description": "",  
12  "dependencies": {  
13    "cors": "^2.8.5",  
14    "express": "^5.1.0",  
15    "mysql2": "^3.14.1"  
16  }  
17 }  
18
```

A red arrow points to the 'cors' dependency in the 'dependencies' object.

Below the code editor, the 'TERMINAL' tab is active, showing the output of the command 'npm i cors':

```
found 0 vulnerabilities  
  
C:\Users\Aprendiz\Documents\fritosoftapi>npm i cors  
  
added 2 packages, and audited 80 packages in 1s  
  
15 packages are looking for funding  
  run 'npm fund' for details  
  
found 0 vulnerabilities  
  
C:\Users\Aprendiz\Documents\fritosoftapi>
```

A red arrow points to the command 'npm i cors' in the terminal output.

En Node.js, bcrypt es una biblioteca para realizar el hashing de contraseñas, lo que significa que convierte una contraseña en texto plano en una cadena de caracteres aleatoria y única, que no puede ser deshecha, para guardarla de forma segura en una base de datos. Es una herramienta fundamental para mejorar la seguridad de las aplicaciones Node.js al proteger las contraseñas de los usuarios.

Código npm i bcrypt --save

```
found 0 vulnerabilities

C:\Users\Aprendiz\Documents\fritosoftapi>npm i bcrypt --save

added 3 packages, and audited 83 packages in 3s

15 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

dotenv en Node.js es una biblioteca que facilita la gestión de variables de entorno. Permite cargar variables de entorno desde un archivo .env (que contiene pares clave-valor) a la variable global process.env. Esto es útil para guardar configuraciones y secretos de la aplicación (como claves de API o contraseñas) de forma segura, separadas del código fuente.

Código npm i dotenv --save

```
C:\Users\Aprendiz\Documents\fritosoftapi>npm i dotenv --save

added 1 package, and audited 84 packages in 1s

16 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Así debe quedar packen.json

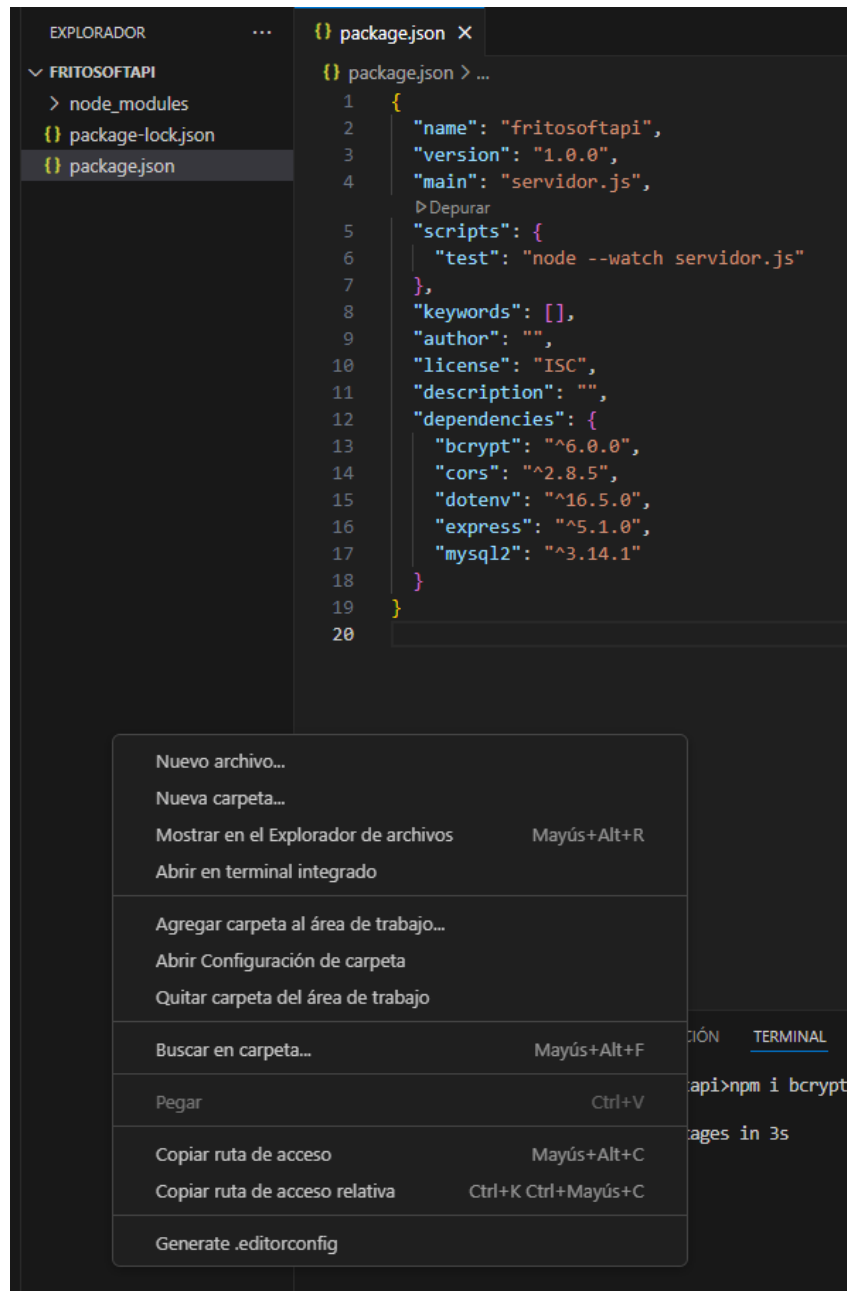
```
{ } package.json X
{ } package.json > ...
1  {
2    "name": "fritosoftapi",
3    "version": "1.0.0",
4    "main": "servidor.js",
5    "scripts": {
6      "test": "node --watch servidor.js"
7    },
8    "keywords": [],
9    "author": "",
10   "license": "ISC",
11   "description": "",
12   "dependencies": {
13     "bcrypt": "^6.0.0",
14     "cors": "^2.8.5",
15     "dotenv": "^16.5.0",
16     "express": "^5.1.0",
17     "mysql2": "^3.14.1"
18   }
19 }
20
```

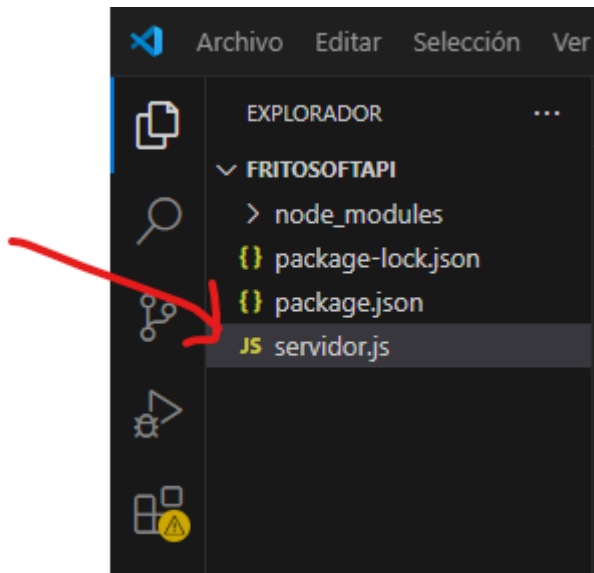

5 paso

Crear el archivo principal o main que se configuro el
package.json

Se llama servidor.js (fuera del archivo crear el archivo)

Nuevo archivo





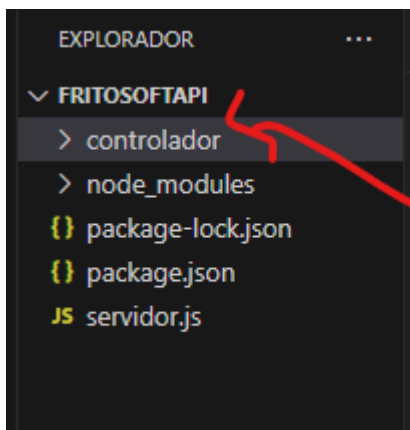
Así debe quedar el archivo main.

Ahora vamos a crear la arquitectura MVC, carpetas controlador - modelo – vista

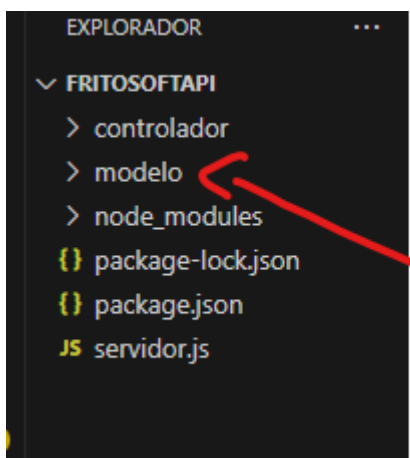
6 paso

Vamos a crear MVC

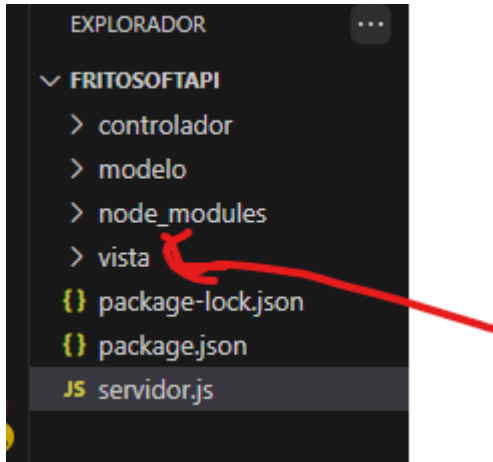
La arquitectura MVC (Modelo-Vista-Controlador) es un patrón de diseño de software que divide una aplicación en tres componentes interconectados: el Modelo, la Vista y el Controlador. Este patrón de diseño promueve la separación de preocupaciones, lo que facilita la gestión de la lógica de negocio, la interfaz de usuario y el flujo de control.



Modelo

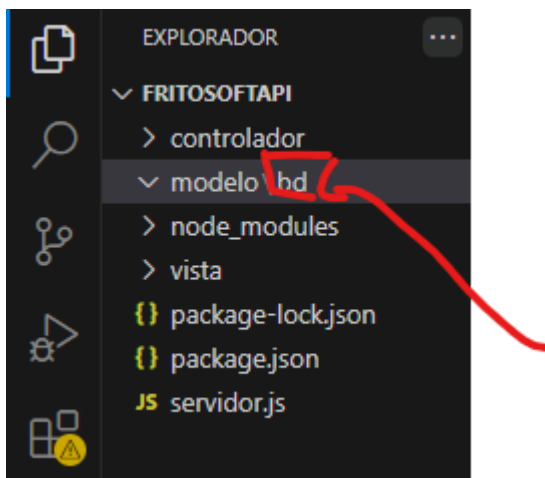


Vista



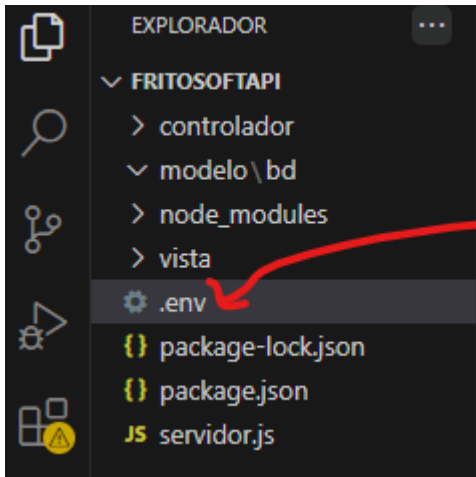
Ahora definimos que en modelo va dentro otra carpeta que separa la conexión de los microservicios

Vamos a crear bd

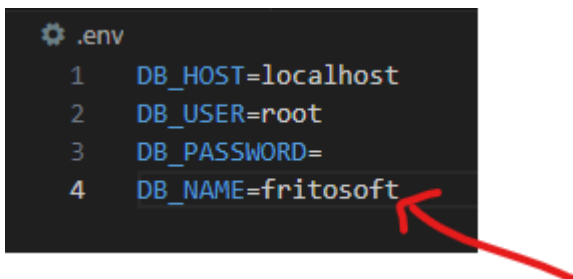


Paso 7

Vamos a crear un archivo `.env` entorno del sistema para que los datos sensibles queden protegidos y no expuestos por este archivo



Vamos a crear los códigos

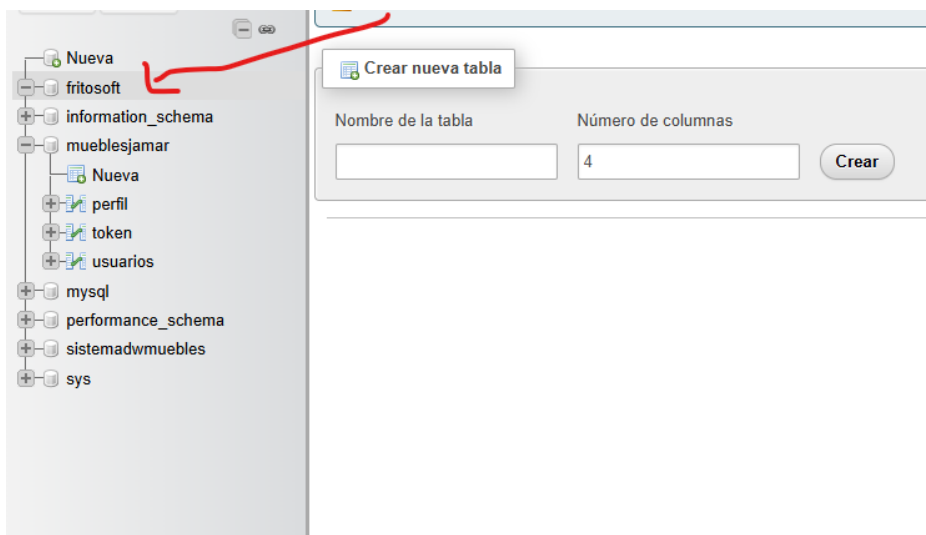


Línea 1 para el nombre del servidor como estamos en local por eso el servidor es localhost

Línea 2 es para el nombre del usuario del servidor y el usuario por defecto es root

Línea 3 es para la contraseña como no tiene por eso vacía.

Línea 4 es para el nombre de la base datos



Paso 8

Crear en la carpeta bd los 2 archivos para la conexión a la base datos.

```
CREATE TABLE `perfil` (  
  `idPerfil` int NOT NULL,  
  `documento` varchar(10) COLLATE utf8mb4_general_ci NOT NULL,  
  `nombres` varchar(100) COLLATE utf8mb4_general_ci NOT NULL,  
  `telefono` varchar(10) COLLATE utf8mb4_general_ci NOT NULL,  
  `correo` varchar(200) COLLATE utf8mb4_general_ci NOT NULL,  
  `respuesta1` varchar(255) COLLATE utf8mb4_general_ci DEFAULT NULL,  
  `respuesta2` varchar(255) COLLATE utf8mb4_general_ci DEFAULT NULL,  
  `respuesta3` varchar(255) COLLATE utf8mb4_general_ci DEFAULT NULL,  
  `direccion` varchar(255) COLLATE utf8mb4_general_ci DEFAULT NULL,  
  `fechaexpedicion` varchar(30) COLLATE utf8mb4_general_ci DEFAULT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;  
  
-----  
  
CREATE TABLE `token` (  
  `idToken` int NOT NULL,  
  `usuario` varchar(100) COLLATE utf8mb4_general_ci NOT NULL,  
  `rol` varchar(20) COLLATE utf8mb4_general_ci NOT NULL,  
  `correo` varchar(200) COLLATE utf8mb4_general_ci NOT NULL,  
  `llave` varchar(255) COLLATE utf8mb4_general_ci NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;  
  
--  
-- Volcado de datos para la tabla `token`  
--  
  
INSERT INTO `token` (`idToken`, `usuario`, `rol`, `correo`, `llave`) VALUES  
(1, 'pepito perez', 'Cliente', 'pepito@gmail.com', 'da8c7603ecc7fde59e137685e1ce8501c31759d0ca14ee42cf3d6452b4770099');  
  
-----
```

```

CREATE TABLE `token` (
  `idToken` int NOT NULL,
  `usuario` varchar(100) COLLATE utf8mb4_general_ci NOT NULL,
  `rol` varchar(20) COLLATE utf8mb4_general_ci NOT NULL,
  `correo` varchar(200) COLLATE utf8mb4_general_ci NOT NULL,
  `llave` varchar(255) COLLATE utf8mb4_general_ci NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Volcado de datos para la tabla `token`
--

INSERT INTO `token` (`idToken`, `usuario`, `rol`, `correo`, `llave`) VALUES
(1, 'pepito perez', 'Cliente', 'pepito@gmail.com', 'da8c7603ecc7fde59e137685e1ce8501c31759d0ca14ee42cf3d6452b4770099');

CREATE TABLE `usuarios` (
  `idUsuario` int NOT NULL,
  `documento` varchar(10) CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci NOT NULL,
  `nombres` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci NOT NULL,
  `telefono` varchar(10) CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci NOT NULL,
  `correo` varchar(200) CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci NOT NULL,
  `contrasena` varchar(255) CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci NOT NULL,
  `rol` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci NOT NULL DEFAULT 'Cliente',
  `estado` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci NOT NULL DEFAULT 'Activo',
  `fechaCreacion` varchar(30) COLLATE utf8mb4_general_ci NOT NULL,
  `intentosFallidos` int NOT NULL DEFAULT '0'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Volcado de datos para la tabla `usuarios`
--

INSERT INTO `usuarios` (`idUsuario`, `documento`, `nombres`, `telefono`, `correo`, `contrasena`, `rol`, `estado`, `fechaCreacion`, `intentosFallidos`) VALUES
(1, '123456789', 'pepito perez', '3011111111', 'pepito@gmail.com', '$2b$10$vF18uP78.ZjZAB7vQuusWe71N29UDdZyKU73wVpZkVo53p.7gLuv6', 'Cliente', 'Activo', '2025-05-16 23:58:56.786', 0),
(2, '123456788', 'juanita perez', '3011111112', 'juanita@gmail.com', '$2b$10$07ur/hnKJw8s8hvn9jNLweXFV62N0sGS5jiHmYKh6oy3hvf5AnJe2', 'Cliente', 'Activo', '2025-05-17 01:37:50.129', 1);

```

```

ALTER TABLE `perfil`
  ADD PRIMARY KEY (`idPerfil`);

--
-- Indices de la tabla `token`
--

ALTER TABLE `token`
  ADD PRIMARY KEY (`idToken`),
  ADD UNIQUE KEY `usuario` (`usuario`),
  ADD UNIQUE KEY `correo` (`correo`);

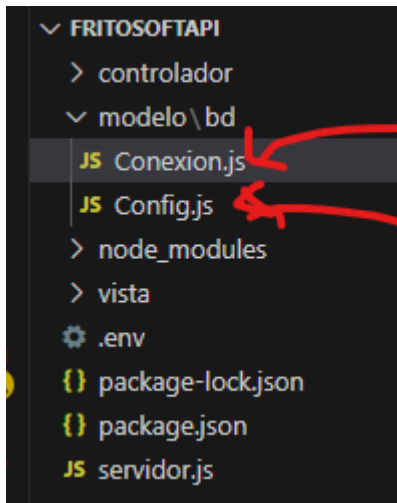
--
-- Indices de la tabla `usuarios`
--

ALTER TABLE `usuarios`
  ADD PRIMARY KEY (`idUsuario`),
  ADD UNIQUE KEY `documento` (`documento`),
  ADD UNIQUE KEY `telefono` (`telefono`),
  ADD UNIQUE KEY `correo` (`correo`);

```

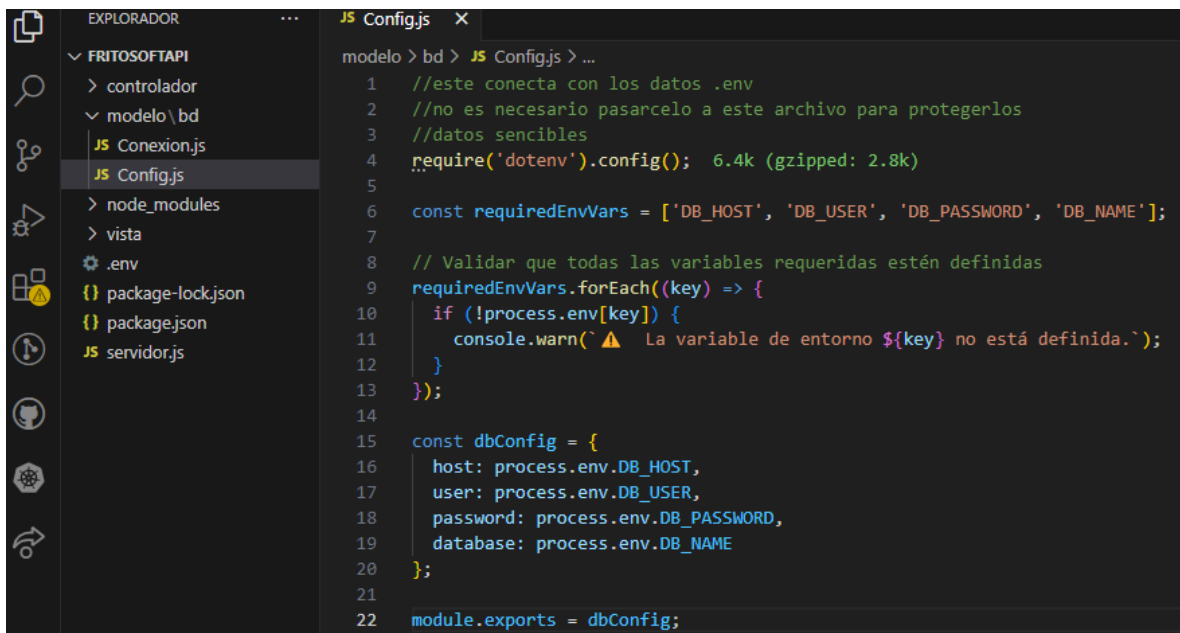


```
--  
  
ALTER TABLE `usuarios`  
  MODIFY `idUsuario` int NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=3;  
  
COMMIT;
```

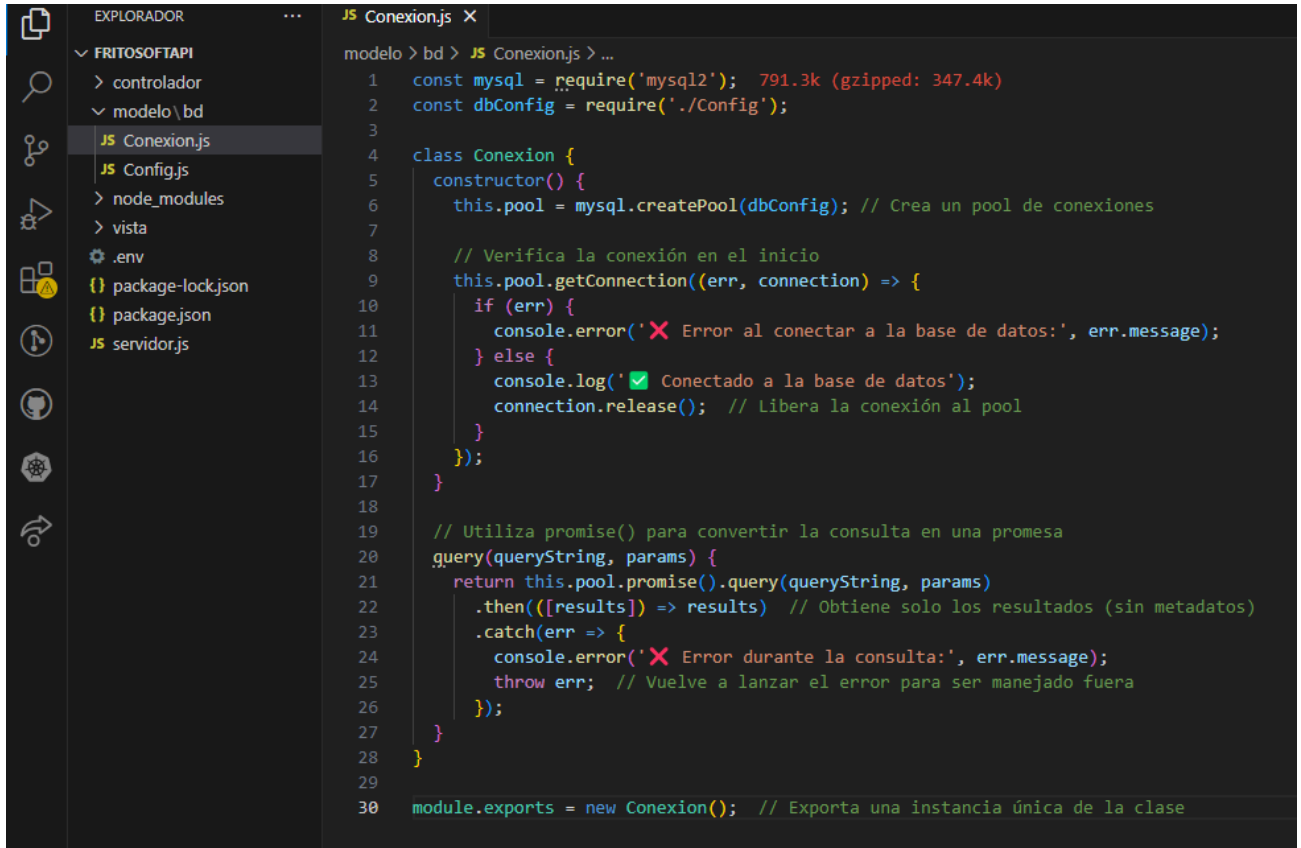


Creado los 2 archivos

Archivo Config.js



Crear el Conexión.js



```
1  const mysql = require('mysql2');
2  const dbConfig = require('./Config');
3
4  class Conexion {
5    constructor() {
6      this.pool = mysql.createPool(dbConfig); // Crea un pool de conexiones
7
8      // Verifica la conexión en el inicio
9      this.pool.getConnection((err, connection) => {
10        if (err) {
11          console.error('❌ Error al conectar a la base de datos:', err.message);
12        } else {
13          console.log('✅ Conectado a la base de datos');
14          connection.release(); // Libera la conexión al pool
15        }
16      });
17    }
18
19    // Utiliza promise() para convertir la consulta en una promesa
20    query(queryString, params) {
21      return this.pool.promise().query(queryString, params)
22        .then(([results]) => results) // Obtiene solo los resultados (sin metadatos)
23        .catch(err => {
24          console.error('❌ Error durante la consulta:', err.message);
25          throw err; // Vuelve a lanzar el error para ser manejado fuera
26        });
27    }
28  }
29
30  module.exports = new Conexion(); // Exporta una instancia única de la clase
```

Con esto ya queda la conexión a la base datos.



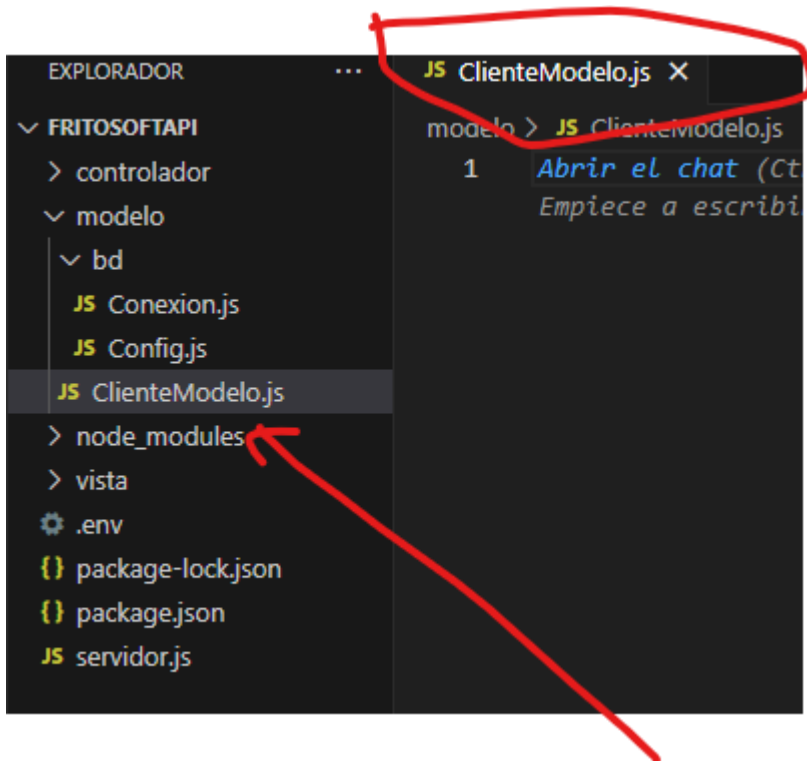
Que contengan la palabra:

Tabla	Acción	Filas	Tipo	Cotejamiento	Tamaño	Residuo a depurar
☐ perfil	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_general_ci	16.0 KB	-
☐ token	Examinar Estructura Buscar Insertar Vaciar Eliminar	1	InnoDB	utf8mb4_general_ci	16.0 KB	-
☐ usuarios	Examinar Estructura Buscar Insertar Vaciar Eliminar	2	InnoDB	utf8mb4_general_ci	16.0 KB	-
3 tablas Número de filas		3	InnoDB	utf8mb4_general_ci	48.0 KB	0 B

☐ Seleccionar todo Para los elementos que están marcados: ▼

Paso 9

Vamos a crear el microservicio para resolver el rf-01 crear cuenta de cliente siendo este el primer paso para resolver rf



Ahora vamos a escribir el código

```
JS ClienteModelo.js X
modelo > JS ClienteModelo.js
1  Abrir el chat (Ct
Empiece a escribi

const dbService = require('../bd/Conexion');
const bcrypt = require('bcrypt'); 7.2k (gzipped: 2.5k)

class ClienteModelo {
  // funcion para crear nuevos clientes
  static async crearClientes(doc, name, tel, email, contras) {
    const query = 'INSERT INTO usuarios (documento, nombres, telefono, correo, contrasena, fechaCreacion) VALUES (?, ?, ?, ?, ?, ?)';

    try {
      // Generar el hash de la contraseña con bcrypt
      const salto = 10; // Nivel de seguridad de encriptación
      const contra = await bcrypt.hash(contras, salto);

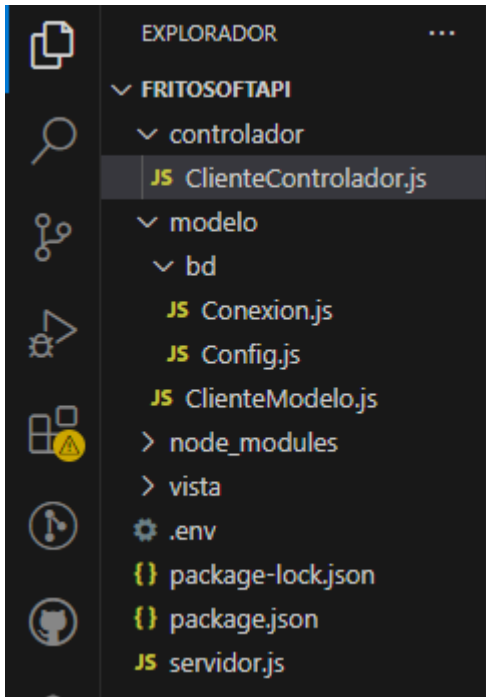
      return await dbService.query(query, [doc, name, tel, email, contra, new Date()]);
    } catch (err) {
      throw new Error('Error al crear su nueva cuenta: ${err.message}');
    }
  }
}

module.exports = ClienteModelo;
```

Paso 9

Ahora vamos a crear segundo paso para resolver rf crear el archivo de controlador que es que resuelve y da respuesta a la petición del usuario final

Llamado ClienteControlador.js



Ahora veremos el código

JS ClienteControlador.js X

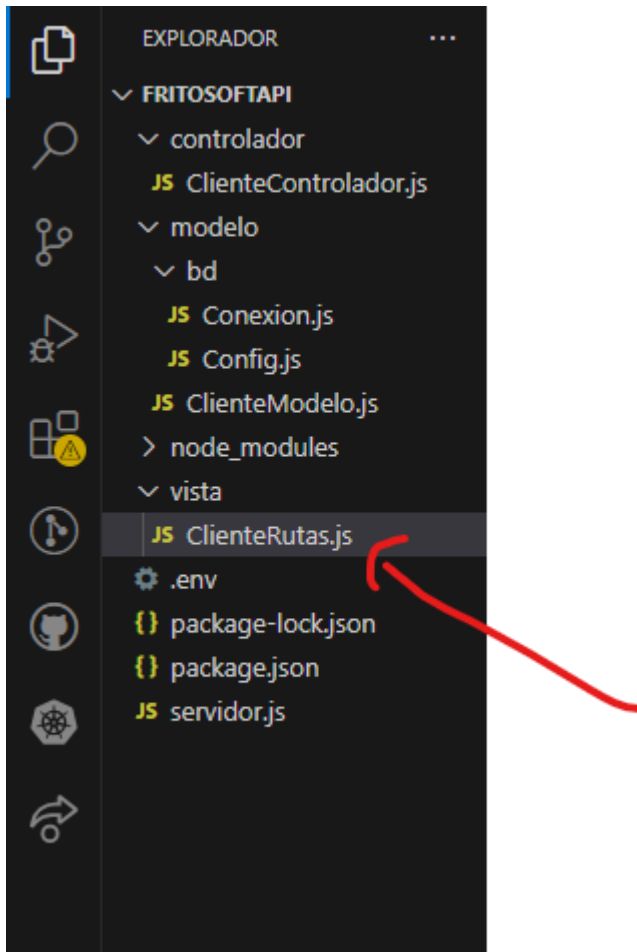
controlador > JS ClienteControlador.js > ...

```
1  const modelo = require('../modelo/ClienteModelo');
2
3  class ClienteControlador {
4    // funcion crear nuevo cliente
5    static async crearCliente(req, res) {
6      const { t1: doc, t2: name, t3: tel, t4: email, t5: contra } = req.body;
7      // ----- o validaciones o -----
8      // Validar campos vacíos ? ? ? ? ? -----
9      const errorCampos = ClienteControlador.verCampos(doc, name, tel, email, contra);
10     if (errorCampos) {
11       return res.status(400).json({ error: errorCampos });
12     }
13     // Validar documento ? ? ? ? ? -----
14     const erorIde = ClienteControlador.verIde(doc);
15     if (erorIde) {
16       return res.status(400).json({ error: erorIde });
17     }
18     // Validar nombres completos ? ? ? ? ? -----
19     const errornom = ClienteControlador.vernom(name);
20     if (errornom) {
21       return res.status(400).json({ error: errornom });
22     }
23     // Validar teléfono ? ? ? ? ? -----
24     const errortel = ClienteControlador.verTel(tel);
25     if (errortel) {
26       return res.status(400).json({ error: errortel });
27
28     // Validar correo ? ? ? ? ? -----
29     const errorem = ClienteControlador.veremail(email);
30     if (errorem) {
31       return res.status(400).json({ error: errorem });
32     }
33     // Validar contraseña ? ? ? ? ? -----
34     const errorkey = ClienteControlador.verkey(contra);
35     if (errorkey) {
36       return res.status(400).json({ error: errorkey });
37     }
38
39     try {
40       const result = await modelo.crearClientes(doc, name, tel, email, contra);
41       res.status(201).json({ mensaje: 'Usuario creado', id: result.insertId });
42     } catch (err) {
43       if (err.message.includes("Duplicate entry")) {
44         return res.status(409).json({ error: 'Ya existe un usuario con estos datos.',
45           sugerencia: 'intenta recuperar la cuenta o inicia sesión.' });
46       } else {
47         return res.status(500).json({ error: 'Error inesperado: ' + err.message });
48       }
49     }
50     // ----- o fin validaciones o -----
51
52     } //cerrar crearcliente-----
```


Paso 10

Ahora vamos a crear el archivo que controla las rutas para cliente

Llamado ClienteRutas.js



Ahora vamos al código

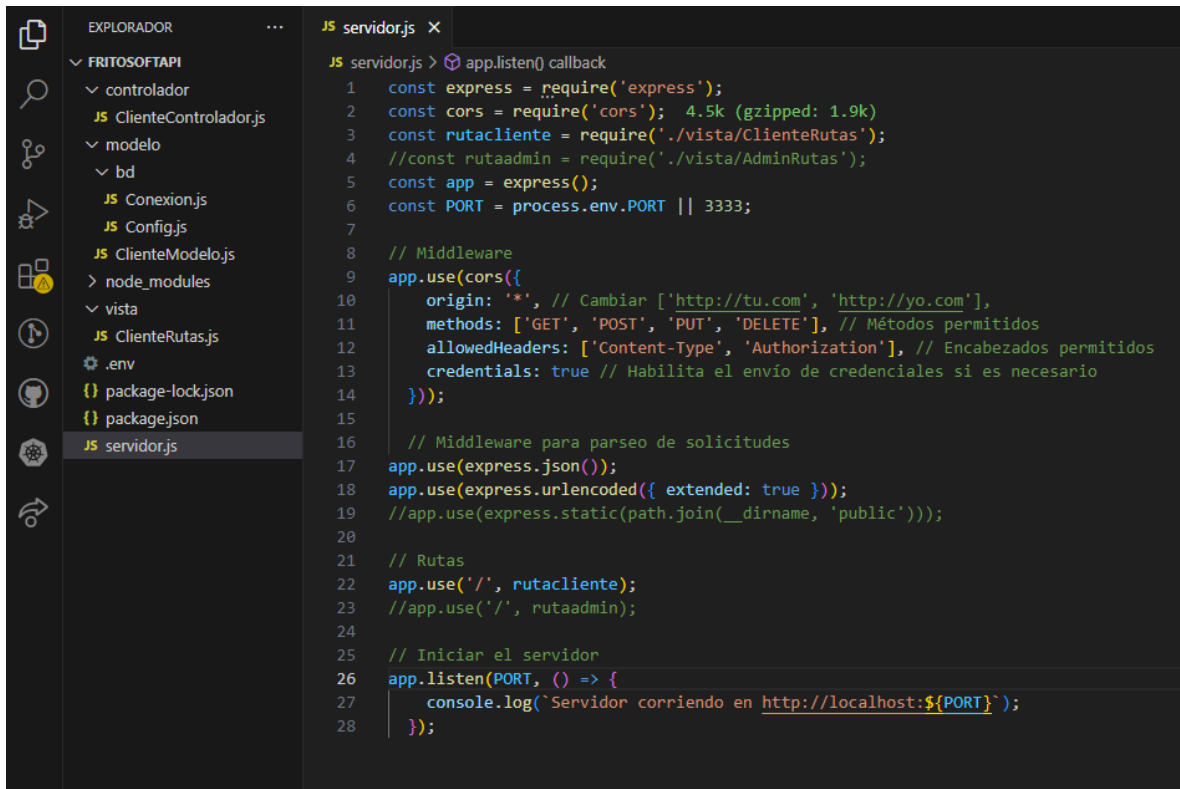
JS ClienteRutas.js X

vista > JS ClienteRutas.js > ...

```
1  const express = require('express');
2  const CRutas = require('../controlador/ClienteControlador');
3  const LRutas = require('../controlador/LoginClienteControlador');
4  const router = express.Router();
5
6  router.post('/usuarios', CRutas.crearCliente);
7  router.post('/login', LRutas.validarCredencial);
8  module.exports = router;
```


Paso 11

Vamos a escribir los códigos del archivo principal y en este llevara rutas llevara el servidor crearlo.



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'FRITOSOFTAPI' with the following structure:

- controlador
 - JS ClienteControlador.js
- modelo
 - bd
 - JS Conexion.js
 - JS Config.js
 - JS ClienteModelo.js
- node_modules
- vista
 - JS ClienteRutas.js
- .env
- package-lock.json
- package.json
- JS servidor.js (selected)

The code editor shows the content of 'servidor.js' with the following code:

```
JS servidor.js > app.listen() callback
1  const express = require('express');
2  const cors = require('cors'); 4.5k (gzipped: 1.9k)
3  const rutacliente = require('./vista/ClienteRutas');
4  //const rutaadmin = require('./vista/AdminRutas');
5  const app = express();
6  const PORT = process.env.PORT || 3333;
7
8  // Middleware
9  app.use(cors({
10     origin: '*', // Cambiar ['http://tu.com', 'http://yo.com'],
11     methods: ['GET', 'POST', 'PUT', 'DELETE'], // Métodos permitidos
12     allowedHeaders: ['Content-Type', 'Authorization'], // Encabezados permitidos
13     credentials: true // Habilita el envío de credenciales si es necesario
14 }));
15
16 // Middleware para parseo de solicitudes
17 app.use(express.json());
18 app.use(express.urlencoded({ extended: true }));
19 //app.use(express.static(path.join(__dirname, 'public')));
20
21 // Rutas
22 app.use('/', rutacliente);
23 //app.use('/', rutaadmin);
24
25 // Iniciar el servidor
26 app.listen(PORT, () => {
27     console.log(`Servidor corriendo en http://localhost:${PORT}`);
28 });
```

Con esto ya tenemos el servidor listo.

Estos códigos los encuentras en el repositorio

<https://github.com/DonLlogui/fritosof>

video que también explica youtube